TREES

Complete GATE DA Study Material

Theory + Properties + Traversals + BST + Heap + PYQ-style Problems

---

1 What is a Tree?

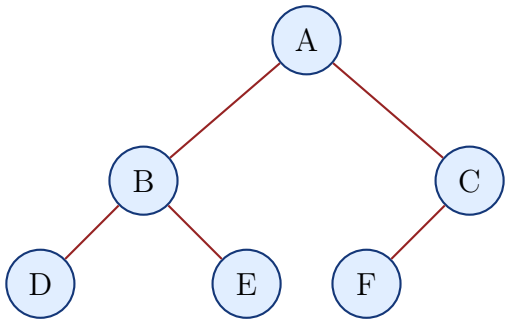
Definition

A **tree** is a non-linear hierarchical data structure consisting of nodes connected by edges.
A tree with n nodes contains exactly:

$$\boxed{n-1 \text{ edges}}$$

for a non-empty tree.

Example:



Here:

$$n = 6$$

and therefore:

$$E = n - 1 = 5$$

2 Tree Terminology

Term	Meaning
Node	An element of the tree
Root	Topmost node
Edge	Connection between two nodes
Parent	Immediate predecessor of a node
Child	Immediate successor of a node
Sibling	Nodes having the same parent
Leaf	Node having no children
Internal node	Node having at least one child
Degree of node	Number of children of the node
Degree of tree	Maximum node degree
Path	Sequence of connected nodes

Ancestor	Parent, grandparent, etc.
Descendant	Child, grandchild, etc.
Subtree	Tree formed by a node and its descendants
Depth	Distance from root
Height	Maximum distance from a node to a leaf

3 Root

The root is the unique node having no parent.

In the diagram:

$$\boxed{A = \text{Root}}$$

The root has depth:

$$\boxed{0}$$

under the common GATE convention.

4 Parent and Child

If:

$$A \to B$$

then A is the parent of B and B is the child of A.

For example:

$$A$$

has children:

$$B, C$$

5 Leaf Node

A leaf node has no children.

For example:

$$D,\ E,\ F$$

are leaves in the earlier tree.

Therefore:

$$\boxed{\text{degree of a leaf} = 0}$$

6 Internal Node

An internal node has at least one child.

For example:

$$A,\ B,\ C$$

are internal nodes in the earlier tree.

7 Depth

Depth of a node is the number of edges from the root to that node.

Root:

$$\boxed{depth(root) = 0}$$

Child of root:

$$\boxed{depth = 1}$$

Grandchild:

$$\boxed{depth = 2}$$

8 Height

Height of a node is the number of edges on the longest path from that node to a leaf.

Therefore:

$$\boxed{height(leaf) = 0}$$

The height of the tree is the height of the root.

$$\boxed{h(T) = height(root)}$$

GATE Important

Some textbooks define height in terms of number of nodes rather than number of edges.
For GATE questions, carefully inspect the definition/convention being used.

9 Fundamental Tree Properties

For a tree with n nodes:

$$\boxed{E = n - 1}$$

For a rooted tree:

$$\boxed{\sum_{v} \deg(v) = n - 1}$$

where degree means number of children in the rooted-tree sense.

For an undirected tree:

$$\boxed{\sum \deg(v) = 2(n-1)}$$

because every edge contributes 2 to the degree sum.

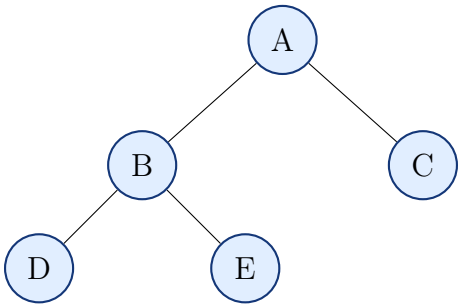
10 Binary Tree

Definition

A **binary tree** is a tree in which every node has at most two children.
The children are called:

$$\boxed{\text{Left Child and Right Child}}$$

Example:



Maximum degree:

$$\boxed{2}$$

11 Maximum Number of Nodes at Level l

For a binary tree with root at level 0:

$$\boxed{\text{Maximum nodes at level } l = 2^l}$$

Examples:

$$l = 0 \Rightarrow 1$$

$$l = 1 \Rightarrow 2$$

$$l = 2 \Rightarrow 4$$

$$l = 3 \Rightarrow 8$$

12 Maximum Number of Nodes in Binary Tree

If height is h measured in edges:

$$1 + 2 + 4 + \cdots + 2^h$$

Therefore:

$$\boxed{N_{\max} = 2^{h+1} - 1}$$

13 Minimum Height of Binary Tree

For n nodes, minimum possible height is:

$$\boxed{\lceil \log_2(n+1) \rceil - 1}$$

because:

$$n \leq 2^{h+1} - 1$$

14 Maximum Height of Binary Tree

A completely skewed binary tree has one node per level.

Therefore:

$$\boxed{h_{\max} = n - 1}$$

15 Full Binary Tree

A full binary tree is one where every node has either:

$$0 \text{ children}$$

or:

$$2 \text{ children.}$$

Let:

$$I = \text{number of internal nodes}$$

and:

$$L = \text{number of leaves.}$$

Then:

$$\boxed{L = I + 1}$$

Total nodes:

$$\boxed{N = 2I + 1}$$

Therefore:

$$\boxed{N = 2L - 1}$$

16 Perfect Binary Tree

A perfect binary tree has:

1. Every internal node having exactly two children.
2. Every leaf at the same level.

For height h:

$$\boxed{N = 2^{h+1} - 1}$$

Leaves:

$$\boxed{L = 2^h}$$

Internal nodes:

$$\boxed{I = 2^h - 1}$$

17 Complete Binary Tree

A complete binary tree has:

1. All levels completely filled except possibly the last.
2. The last level filled from left to right.

GATE Important

Complete binary trees are extremely important for **binary heaps**.

18 Balanced Binary Tree

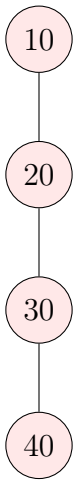
A balanced tree keeps the height approximately logarithmic.
Ideally:

$$\boxed{h = O(\log n)}$$

This leads to efficient searching in ordered trees.

19 Skewed Binary Tree

A skewed tree has nodes concentrated along one side.
Example:



Height:

$$\boxed{n-1}$$

20 Tree Traversals

The three fundamental depth-first traversals are:

1. Preorder
2. Inorder
3. Postorder

There is also:

$$\boxed{\text{Level-order traversal}}$$

21 Preorder Traversal

Rule:

$$\boxed{\text{Root} \rightarrow \text{Left} \rightarrow \text{Right}}$$

Mnemonic:

$$\boxed{RLR}$$

Algorithm:

```
preorder(node):

    if node == NULL:
        return

    visit(node)
    preorder(node.left)
    preorder(node.right)
```

22 Inorder Traversal

Rule:

$$\boxed{\text{Left} \rightarrow \text{Root} \rightarrow \text{Right}}$$

Mnemonic:

$$\boxed{LRR}$$

More clearly:

$$\boxed{LNR}$$

where N means node.
Algorithm:

```
inorder(node):

    if node == NULL:
        return

    inorder(node.left)
    visit(node)
    inorder(node.right)
```

23 Postorder Traversal

Rule:

$$\boxed{\text{Left} \rightarrow \text{Right} \rightarrow \text{Root}}$$

Mnemonic:

$$\boxed{LRN}$$

Algorithm:

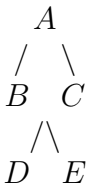
```
postorder(node):

    if node == NULL:
        return

    postorder(node.left)
    postorder(node.right)
    visit(node)
```

24 Level Order Traversal

Level order visits nodes level by level.
It uses a queue.
Example:



Level order:

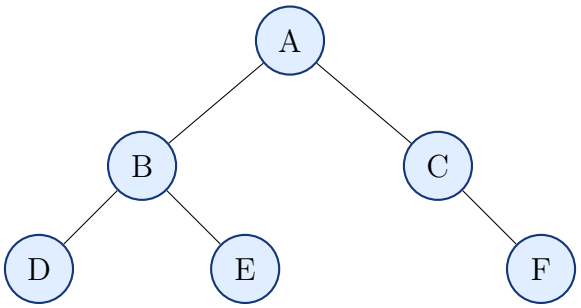
$$\boxed{A, B, C, D, E}$$

Complexity:

$$\boxed{O(n)}$$

25 Traversal Example

Consider:



Preorder:

$$\boxed{A, B, D, E, C, F}$$

Inorder:

$$\boxed{D, B, E, A, C, F}$$

Postorder:

$$\boxed{D, E, B, F, C, A}$$

Level order:

$$\boxed{A, B, C, D, E, F}$$

26 Traversal Complexity

Every node is visited once.

Therefore:

$$\boxed{O(n)}$$

for preorder, inorder, postorder, and level order.

Recursive DFS auxiliary space:

$$\boxed{O(h)}$$

where h is tree height.
For a balanced tree:

$$O(h) = O(\log n)$$

For a skewed tree:

$$O(h) = O(n)$$

27 Binary Search Tree

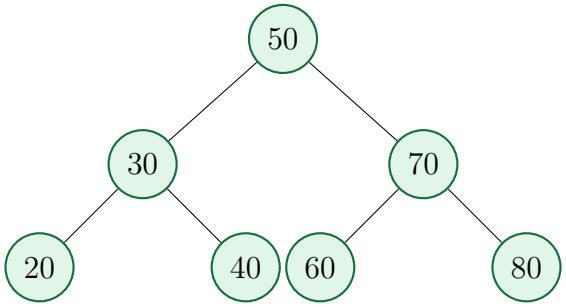
Definition

A Binary Search Tree (BST) is a binary tree satisfying:

$$\boxed{\text{All keys in left subtree} < \text{root} < \text{all keys in right subtree}}$$

for the usual distinct-key convention.

Example:



28 BST Search

For key x:

$$x < root \Rightarrow \text{go left}$$

$$x > root \Rightarrow \text{go right}$$

$$x = root \Rightarrow \text{found}$$

Complexity:
Balanced BST:

$$\boxed{O(\log n)}$$

Worst-case skewed BST:

$$\boxed{O(n)}$$

29 BST Insertion

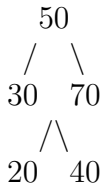
To insert a key:

1. Compare with root.
2. Move left if smaller.
3. Move right if larger.
4. Insert at the first NULL position.

Example insert:

$$50, 30, 70, 20, 40$$

produces:



30 BST Important Property

Inorder traversal of a BST gives sorted order.

Therefore:

$$\boxed{\text{BST inorder} = \text{ascending sorted sequence}}$$

Example:

$$20, 30, 40, 50, 60, 70, 80$$

31 BST Deletion

There are three cases.

31.1 Case 1: Leaf

Simply remove the node.

31.2 Case 2: One Child

Replace the node by its child.

31.3 Case 3: Two Children

Replace the node by:

$$\boxed{\text{Inorder successor}}$$

or:

$$\boxed{\text{Inorder predecessor}}$$

Then delete the replacement node from its original location.

32 Inorder Successor

For a node having a right subtree:

$$\boxed{\text{Successor} = \text{minimum node in right subtree}}$$

33 Inorder Predecessor

For a node having a left subtree:

$$\boxed{\text{Predecessor} = \text{maximum node in left subtree}}$$

34 BST Complexity

Operation	Balanced	Worst Case
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$

35 Binary Heap

A binary heap is a:

$$\boxed{\text{Complete Binary Tree}}$$

satisfying the heap-order property.
Two types:

1. Max Heap
2. Min Heap

36 Max Heap

Every parent is greater than or equal to its children.

$$\boxed{parent \geq child}$$

Therefore the maximum element is at the root.

37 Min Heap

Every parent is less than or equal to its children.

$$\boxed{parent \leq child}$$

Therefore the minimum element is at the root.

38 Array Representation of Heap

For 0-based indexing:

$$\boxed{left(i) = 2i + 1}$$

$$\boxed{right(i) = 2i + 2}$$

$$\boxed{parent(i) = \left\lfloor \frac{i-1}{2} \right\rfloor}$$

for $i > 0$.
For 1-based indexing:

$$\boxed{left(i) = 2i}$$

$$\boxed{right(i) = 2i + 1}$$

$$\boxed{parent(i) = \left\lfloor \frac{i}{2} \right\rfloor}$$

39 Heap Operations

Insertion:

$$\boxed{O(\log n)}$$

Delete root:

$$\boxed{O(\log n)}$$

Peek root:

$$\boxed{O(1)}$$

Build heap:

$$\boxed{O(n)}$$

Heap sort:

$$\boxed{O(n \log n)}$$

40 Heap vs BST

Property	BST	Heap
Shape	Not necessarily complete	Complete
Root	Not necessarily min/max	Min/max
Inorder sorted?	Yes	No
Search arbitrary key	Efficient if balanced	$O(n)$ generally
Insert	$O(\log n)$ balanced	$O(\log n)$
Delete root	Variable	$O(\log n)$

41 Reconstructing Binary Tree

A binary tree can be uniquely reconstructed from:

$$\boxed{\text{Preorder} + \text{Inorder}}$$

when node keys are distinct.
Also:

$$\boxed{\text{Postorder} + \text{Inorder}}$$

is sufficient.
But generally:

$$\boxed{\text{Preorder} + \text{Postorder alone is NOT sufficient}}$$

to uniquely determine an arbitrary binary tree.

42 Preorder + Inorder Reconstruction

Rules:

1. First element of preorder is root.
2. Locate root in inorder.
3. Elements to left form left subtree.
4. Elements to right form right subtree.
5. Recursively repeat.

43 Solved Reconstruction Example

Given:

$$\text{Preorder} = A, B, D, E, C, F$$

$$\text{Inorder} = D, B, E, A, C, F$$

First preorder element:

$$A$$

Therefore:

$$A = \text{root}$$

In inorder:

$$D, B, E \mid A \mid C, F$$

Thus:

$$\text{Left subtree} = \{D, B, E\}$$

$$\text{Right subtree} = \{C, F\}$$

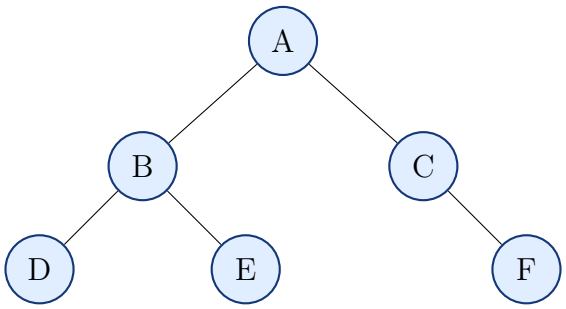
Left subtree preorder:

$$B, D, E$$

Right subtree preorder:

$$C, F$$

Therefore tree:



Postorder:

$$\boxed{D, E, B, F, C, A}$$

44 GATE DA 2026 PYQ – Tree Reconstruction

GATE Important

Real GATE DA 2026 PYQ topic:
The 2026 DA paper provided preorder and inorder traversals of a binary tree and asked candidates to determine which statements about the root, postorder traversal, children, and subtrees are true.
This is an extremely important pattern for GATE DA.

Practice a similar problem:

$$\text{Preorder} = P, Q, S, E, R, F, G$$

$$\text{Inorder} = S, Q, E, P, F, R, G$$

First:

$$\boxed{P = \text{root}}$$

Split inorder:

$$S, Q, E \mid P \mid F, R, G$$

Therefore:

$$\text{Left subtree} = \{S, Q, E\}$$

$$\text{Right subtree} = \{F, R, G\}$$

Construct recursively.
Postorder:

$$\boxed{S, E, Q, F, G, R, P}$$

Thus the following are true:

$$\boxed{\text{P is root}}$$

and:

$$\boxed{\text{Postorder} = S, E, Q, F, G, R, P}$$

This is based on the actual 2026 DA PYQ structure. The official GATE 2026 paper contains this traversal question.

45 Python Binary Tree

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```

46 Python Traversals

```python
def preorder(root):
    if root is None:
        return

    print(root.data, end=" ")
    preorder(root.left)
    preorder(root.right)


def inorder(root):
    if root is None:
        return

    inorder(root.left)
```

```
    print(root.data, end=" ")
    inorder(root.right)


def postorder(root):
    if root is None:
        return

    postorder(root.left)
    postorder(root.right)
    print(root.data, end=" ")
```

47 Solved Numerical Questions

Solved Example

Q1. A binary tree has 15 nodes. What is the maximum possible height?
For a skewed tree:

$$h = n - 1$$

Therefore:

$$\boxed{h = 14}$$

Solved Example

Q2. What is the maximum number of nodes in a binary tree of height 4?

$$N_{\max} = 2^{h+1} - 1$$

$$= 2^5 - 1$$

$$\boxed{31}$$

Solved Example

Q3. A perfect binary tree has height 5. Find the number of leaves.

$$L = 2^h$$

$$L = 2^5$$

$$\boxed{32}$$

Solved Example

Q4. A full binary tree has 20 internal nodes. Find the number of leaves.

$$L = I + 1$$

$$L = 20 + 1$$

$$\boxed{21}$$

Solved Example

Q5. A full binary tree has 25 internal nodes. Find total nodes.

$$N = 2I + 1$$

$$N = 2(25) + 1$$

$$\boxed{51}$$

48 Practice Questions

Practice Question

Q1. A tree has 20 nodes. How many edges does it have?

Practice Question

Q2. What is the maximum number of nodes at level 5 of a binary tree?

Practice Question

Q3. Find the maximum number of nodes in a binary tree of height 6.

Practice Question

Q4. A perfect binary tree has height 4. Find the number of internal nodes.

Practice Question

Q5. A full binary tree has 31 nodes. Find the number of leaves.

Practice Question

Q6. For the BST containing:

$$50, 30, 70, 20, 40, 60, 80$$

write its inorder traversal.

Practice Question

Q7. For the same BST, find the inorder successor of 50.

Practice Question

Q8. Given:

$$\text{Preorder} = A, B, D, E, C, F$$

$$\text{Inorder} = D, B, E, A, C, F$$

find the postorder traversal.

Practice Question

Q9. Which traversal of a BST produces sorted order?

Practice Question

Q10. A binary heap contains 31 elements. What is its minimum possible height?

49 Answers

Important Formula

1. 19
2. $2^5 = 32$
3. $2^7 - 1 = 127$
4. $2^4 - 1 = 15$
5. 16
6. $20, 30, 40, 50, 60, 70, 80$
7. 60
8. D, E, B, F, C, A
9. Inorder
10. 4

50 GATE-Level MCQs

Practice Question

Q11. Which traversal of a BST always produces keys in non-decreasing order?

A) Preorder

B) Inorder

C) Postorder

D) Level order

Answer: B

Practice Question

Q12. A binary tree with n nodes can have at most:

A) n edges

B) $n + 1$ edges

C) $n - 1$ edges

D) $2n$ edges

Answer: C

Practice Question

Q13. Which tree has exactly 2^h leaves when height is h?

A) Arbitrary binary tree

B) Perfect binary tree

C) BST

D) Skewed tree

Answer: B

Practice Question

Q14. The worst-case search complexity of a BST with n nodes is:

A) $O(1)$

B) $O(\log n)$

C) $O(n)$

D) $O(n \log n)$

Answer: C

Practice Question

Q15. Which traversal is naturally implemented using a queue?

A) Preorder

B) Inorder

C) Postorder

D) Level order

Answer: D

51 Most Important Tree Results for GATE

Important Formula

$$\boxed{E = n - 1}$$

$$\boxed{\text{Max nodes at level } l = 2^l}$$

$$\boxed{N_{\max} = 2^{h+1} - 1}$$

$$\boxed{h_{\max} = n - 1}$$

$$\boxed{L = I + 1 \quad \text{(full binary tree)}}$$

$$\boxed{N = 2I + 1 \quad \text{(full binary tree)}}$$

$$\boxed{L = 2^h \quad \text{(perfect binary tree)}}$$

$$\boxed{\text{BST inorder} = \text{sorted order}}$$

$$\boxed{\text{Balanced BST operations} = O(\log n)}$$

$$\boxed{\text{Worst BST operations} = O(n)}$$

$$\boxed{\text{Heap insertion} = O(\log n)}$$

$$\boxed{\text{Heap root access} = O(1)}$$

$$\boxed{\text{Build heap} = O(n)}$$

$$\boxed{\text{Heap sort} = O(n \log n)}$$

52 30-Second GATE Revision

GATE Important

Remember:

$$\boxed{\text{Preorder} = Root, Left, Right}$$

$$\boxed{\text{Inorder} = Left, Root, Right}$$

$$\boxed{\text{Postorder} = Left, Right, Root}$$

$$\boxed{\text{Level order} = BFS}$$

$$\boxed{\text{BST inorder is sorted}}$$

$$\boxed{\text{Complete binary tree} \Rightarrow \text{Heap candidate}}$$

$$\boxed{\text{Preorder+Inorder} \Rightarrow \text{unique tree}}$$

$$\boxed{\text{Postorder+Inorder} \Rightarrow \text{unique tree}}$$

$$\boxed{\text{Preorder+Postorder alone} \not\Rightarrow \text{unique arbitrary tree}}$$